

Suvenyrai

Amaru perka suvenyrus užsienio parduotuvėje. Yra N suvenyrų **rūšių**. Parduotuvėje kiekvienos rūšies suvenyrų yra be galo daug.

Kiekvienos rūšies suvenyrų kaina yra fiksuota: i -osios ($0 \leq i < N$) rūšies suvenyro kaina yra $P[i]$ monetų, kur $P[i]$ yra teigiamas sveikasis skaičius.

Amaru žino, kad suvenyrų rūšys yra surikiuotos jų kainų mažėjimo tvarka ir kad tos kainos yra skirtingos: $P[0] > P[1] > \dots > P[N-1] > 0$. Amaru taip pat pavyko sužinoti $P[0]$ vertę. Deja, jis neturi jokios kitos informacijos apie suvenyrų kainas.

Amaru pirks suvenyrus atlikdamas sandorius su pardavėju.

Kiekvienas sandoris susideda iš šių žingsnių:

1. Amaru pardavėjui paduoda tam tikrą teigiamą monetų skaičių.
2. Šias monetas pardavėjas pasideda už prekystalio į krūvelę taip, kad Amaru jų nematytų.
3. Tuomet pardavėjas peržiūri kiekvieną suvenyrų rūšį vieną po kito eilės tvarka: $0, 1, \dots, N-1$. Kiekviena rūšis vieno sandorio metu peržiūrima **lygiai vieną kartą**.
 - Kai peržiūri i -ąją suvenyrų rūšį, jeigu tuo metu krūvelėje yra bent $P[i]$ monetų, tuomet
 - pardavėjas pasiima $P[i]$ monetų iš krūvelės ir
 - pardavėjas atideda vieną i -osios rūšies suvenyrą už prekystalio.
4. Galiausiai, pardavėjas visas likusias monetas ir visus suvenyrus nuo prekystalio atiduoda Amaru.

Prieš pradėdant sandorį už prekystalio nėra nei monetų, nei suvenyrų.

Nurodykite, kokius sandorius turi atlikti Amaru, kad:

- kiekvieno sandorio metu jis nusipirktų **bent vieną** suvenyrą;
 - kiekvienam i tokiu, kad $0 \leq i < N$, jis nusipirktų **lygiai** i suvenyrų, kurie yra i -osios rūšies.
- Atkreipkite dėmesį, kad tai reiškia, jog Amaru neturėtų pirkti nė vieno rūšies 0 suvenyro.

Amaru neprivalo minimizuoti sandorių skaičiaus. Taip pat Amaru turi be galo daug monetų.

Realizacija

Parašykite procedūrą:

```
void buy_souvenirs(int N, long long P0)
```

- N : suvenyrų rūšių kiekis.
- $P0$: skaičiaus $P[0]$ vertė.
- Ši procedūra iškviečiama lygiai vieną kartą kiekvienam testui.

Aukščiau esanti procedūra gali iškviesti šią procedūrą, kad nurodytų Amaru atlikti sandorį:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : monetų kiekis, kurį Amaru paduoda pardavėjui.
- Procedūra grąžina porą. Pirmasis poros elementas yra masyvas L , kuriame didėjimo tvarka saugomos nupirktų suvenyrų rūšys. Antrasis poros elementas yra sveikasis skaičius R , nurodantis gražą (monetų skaičių), kurią Amaru gavo po sandorio.
- Reikalaujama, kad $P[0] > M \geq P[N - 1]$. Sąlyga $P[0] > M$ užtikrina, kad Amaru nenusipirktų nė vieno rūšies 0 suvenyro, o sąlyga $M \geq P[N - 1]$ užtikrina, kad nusipirktų bent vieną suvenyrą. Jei šios sąlygos netenkinamos, bus pateiktas pranešimas `Output isn't correct: Invalid argument`. Atkreipkite dėmesį, kad, priešingai nei $P[0]$ vertė, $P[N - 1]$ vertė nėra pateikta pradinuose duomenyse.
- Procedūra gali būti iškviesta ne daugiau nei 5 000 kartų kiekvienam testui.

Vertinimo programa **nėra adaptyvi**. Tai reiškia, kad kainų seka P yra fiksuota prieš iškviečiant `buy_souvenirs`.

Ribojimai

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ kiekvienam i , kur $0 \leq i < N$.
- $P[i] > P[i + 1]$ kiekvienam i , kur $0 \leq i < N - 1$.

Dalinės užduotys

Dalinė užduotis	Taškai	Papildomi ribojimai
1	4	$N = 2$
2	3	$P[i] = N - i$ kiekvienam i , kur $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ kiekvienam i , kur $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ kiekvienam i , kur $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ kiekvienam i , kur $0 \leq i < N - 1$.
6	33	Papildomų ribojimų nėra.

Pavyzdys

Panagrinėkime tokį iškvietimą.

```
buy_souvenirs(3, 4)
```

Yra $N = 3$ suvenyrų rūšių ir $P[0] = 4$.

Atkreipkite dėmesį, kad galimos tik trys kainų sekos P : $[4, 3, 2]$, $[4, 3, 1]$ ir $[4, 2, 1]$.

Tarkime, kad `buy_souvenirs` iškviečia `transaction(2)`. Tegu šis iškvietimas grąžina $([2], 1)$. Tai reiškia, kad Amaru nusipirko vieną 2-osios rūšies suvenyrą ir pardavėjas grąžino 1-ą monetą. Tai reiškia, kad $P = [4, 3, 1]$, kadangi:

- jei būtų $P = [4, 3, 2]$, iškvietimas `transaction(2)` būtų grąžinęs $([2], 0)$;
- jei būtų $P = [4, 2, 1]$, iškvietimas `transaction(2)` būtų grąžinęs $([1], 0)$.

Tada `buy_souvenirs` gali iškviešti `transaction(3)`, kuris grąžina $([1], 0)$. Tai reiškia, kad Amaru nusipirko vieną 1-osios rūšies suvenyrą ir pardavėjas grąžino 0 monetų. Kol kas iš viso Amaru nusipirko vieną 1-osios ir vieną 2-osios rūšių suvenyrus.

Galiausiai `buy_souvenirs` gali iškviešti `transaction(1)`, kuris grąžina $([2], 0)$. Tai reiškia, kad Amaru nusipirko vieną 2-osios rūšies suvenyrą. Vietoj to taip pat buvo galima iškviešti ir `transaction(2)`. Šiuo metu Amaru iš viso turi vieną 1-osios ir du 2-osios rūšių suvenyrus, ko ir yra reikalaujama.

Pavyzdinė vertinimo programa

Pradinių duomenų formatas:

```
N
P[0] P[1] ... P[N-1]
```

Rezultatų formatas:

```
Q[0] Q[1] ... Q[N-1]
```

Čia $Q[i]$ nurodo, kiek i -osios rūšies suvenyrų buvo nupirkta iš viso (kiekvienam i , kur $0 \leq i < N$).